PRACTICAL: 4

Aim: Write a program to implement Shortest Job First (SJF) Preemptive Scheduling for three processes and calculate the total context switches and average waiting time. The processes have burst times 10ns, 20ns, and 30ns, arriving at 0ns, 2ns, and 6ns, respectively

```
Hp@89L1DOMOF3O5T MSYS /c/Users/Hp/Desktop
$ nano SJF.c

Hp@89L1DOMOF3O5T MSYS /c/Users/Hp/Desktop
$ gcc SJF.c -o SJF

Hp@89L1DOMOF3O5T MSYS /c/Users/Hp/Desktop
$ ./SJF
Waiting Times:
P1 = 0 ns
P2 = 8 ns
P3 = 24 ns

Total Context Switches = 2
Average Waiting Time = 10.67 ns
```

```
GNU nano 8.7                                pract4.c
#include <stdio.h>

struct Process {
    int pid;
    int arrival;
    int burst;
    int remaining;
    int completion;
    int waiting;
};

int main() {
    int n = 3;
    struct Process p[3] = {
        {1, 0, 10, 10, 0, 0},
        {2, 2, 20, 20, 0, 0},
        {3, 6, 30, 30, 0, 0}
    };

    int time = 0, completed = 0;
    int prev = -1;
    int contextSwitches = 0;

    while (completed < n) {
        int shortest = -1;
        int minRemaining = 1e9;

        // Find shortest remaining time process
        for (int i = 0; i < n; i++) {
            if (p[i].arrival <= time && p[i].remaining > 0 && p[i].remaining < minRemaining) {
                minRemaining = p[i].remaining;
                shortest = i;
            }
        }

        if (shortest == -1) {
            time++;
            continue;
        }

        // Context switch check
        if (prev != -1 && prev != shortest) {
            contextSwitches++;
        }
        prev = shortest;

        // Execute process
        p[shortest].remaining--;
        time++;

        // Process finished
        if (p[shortest].remaining == 0) {
            completed++;
            p[shortest].completion = time;
            p[shortest].waiting = p[shortest].completion - p[shortest].arrival - p[shortest].burst;
        }
    }

    // Calculate average waiting time
    float avgWait = 0;
    for (int i = 0; i < n; i++) {
        avgWait += p[i].waiting;
    }
    avgWait /= n;

    printf("Total Context Switches = %d\n", contextSwitches);
    printf("Average Waiting Time = %.2f ns\n", avgWait);

    return 0;
}

^G Help        ^O Write Out   ^F Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-] To Bracket  M-Q Previous   ^B Back        ^◂ Prev Word   ^A Home        ^P Prev Line   M-▴ Scroll Up
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy       ^Q Where Was    M-W Next       ^F Forward     ^▸ Next Word   ^E End         ^N Next Line   M-▾ Scroll Down
```